



POLITECNICO

MILANO 1863

Scuola di Ingegneria Industriale e dell'Informazione

Corso di Laurea in Ingegneria Informatica

Documentazione protocollo client-server progetto Ingegneria del Software

Progetto: "Mesos"

Docente:

Cugola Gianpaolo Saverio

Studenti:

Luca Sciarini
Edoardo Sacchi
Leonardo Taccari
Daniel Russo

Anno Accademico 2025-2026

Indice

1	Architettura di Rete	2
1.1	HybridServerNetworkAdapter	2
1.2	SocketServerNetworkAdapter e ClientSocketHandler	2
1.3	RMIServerNetworkAdapter e RMIGameServer	2
2	Meccanismo di Heartbeat (Ping-Pong)	3
2.1	Heartbeat lato server (Socket)	3
2.2	Heartbeat lato client (Socket)	3
2.3	Heartbeat lato RMI	3
3	Sequence Diagram del Protocollo Client-Server	4
3.1	Scenario 1: Connessione al server e gestione delle lobby	4
3.2	Scenario 2: Avvio della partita e primo GameStateUpdate	5
3.3	Scenario 3: Azione di gioco del client	5
3.4	Scenario 4: Fine partita e invio della classifica	5
3.5	Scenario 5: Meccanismo di heartbeat (ping-pong)	6

1 Architettura di Rete

Il sistema adotta un'architettura client-server in cui un unico server è in grado di gestire contemporaneamente connessioni di tipo **Socket TCP** e connessioni di tipo **RMI** (Remote Method Invocation), anche all'interno della stessa partita. Questa scelta consente la massima flessibilità lato client, che può scegliere al momento dell'avvio dell'applicativo, il protocollo di trasporto preferito, senza che la logica di gioco debba conoscere o dipendere da tale scelta.

1.1 HybridServerNetworkAdapter

Il punto d'ingresso lato server per tutta la comunicazione è la classe

- **HybridServerNetworkAdapter**.

Essa aggrega al proprio interno i due adapter specializzati:

- **SocketServerNetworkAdapter**
- **RMIServerNetworkAdapter**

all'avvio (metodo `start()`), lancia in parallelo due thread dedicati, uno per ciascun adapter. In questo modo il server si pone in ascolto simultaneamente sulla porta Socket e sulla porta RMI, senza che i due canali si blocchino reciprocamente.

Quando occorre inviare una notifica di errore/aggiornamento a un insieme di giocatori, è sempre **HybridServerNetworkAdapter** a instradare il messaggio verso il canale corretto in base al protocollo con cui ogni singolo giocatore è connesso, rendendo l'intera comunicazione in uscita trasparente e uniforme per il resto del sistema.

1.2 SocketServerNetworkAdapter e ClientSocketHandler

Il **SocketServerNetworkAdapter** rimane in ascolto su una **ServerSocket** dedicata. Ad ogni nuova connessione accettata viene creato un oggetto **ClientSocketHandler**, che viene eseguito su un thread separato e si occupa di tutta la comunicazione con quel singolo client Socket: riceve i messaggi in arrivo, li deserializza e li instrada al **ServerController**, e al contempo spedisce al client gli eventi di aggiornamento generati dal server.

I messaggi scambiati via Socket sono comandi ed eventi JSON serializzati e deserializzati tramite **ObjectMapper** su stream di testo. Il protocollo applicativo è definito dalle classi nel package `network.messages` (**GameStateUpdateMessage**, **LobbyUpdateMessage**, **RankingUpdateMessage**) e dall'enum **CommunicationProtocol**, che cataloga i tipi di messaggio riconosciuti da entrambi i lati.

1.3 RMIServerNetworkAdapter e RMIGameServer

L'**RMIServerNetworkAdapter** espone al registry RMI l'interfaccia remota **RMIGameServer**, attraverso la quale i client RMI invocano le azioni di gioco come normali chiamate a metodo remoto. Per le notifiche in senso inverso (server → client), il sistema utilizza il meccanismo di **callback RMI**: ogni client RMI registra presso il server un oggetto remoto (**RMIClientConnection**) che il server può invocare direttamente per recapitare aggiornamenti di stato, aggiornamenti di lobby e classifiche finali, errori, ecc...

Lato client, `RMIClientNetworkAdapter` effettua il lookup del registry, ottiene il riferimento a `RMIGameServer` e registra la propria callback. Da quel momento in poi le invocazioni di gioco viaggiano come chiamate RMI sincrone verso il server, mentre gli aggiornamenti arrivano al client in modo asincrono tramite le callback.

2 Meccanismo di Heartbeat (Ping-Pong)

Per rilevare tempestivamente le disconnessioni – sia del client che del server – è stato implementato un servizio di **heartbeat bidirezionale**, con parametri di timeout differenziati a seconda del lato della comunicazione.

2.1 Heartbeat lato server (Socket)

All'avvio di ogni `ClientSocketHandler` viene richiamato il metodo `startHeartbeat()`, che attiva uno scheduler periodico con intervallo di **5 secondi**. Ad ogni tick lo scheduler invia al client un messaggio di tipo `PING` e avvia un controllo sul pong di risposta:

- Se il `PONG` arriva entro **15 secondi**, il ciclo ricomincia e viene inviato il ping successivo.
- Se entro 15 secondi non arriva alcun `PONG`, il server considera il client disconnesso: chiude la connessione Socket e, se la partita è in corso, attraverso l'infrastruttura di rete e il `MatchManager` provoca l'aborto della partita (`abortGame()`), notificando tutti gli altri client connessi (sia Socket che RMI) con un messaggio di errore visualizzato dalla relativa UI (TUI o GUI).

2.2 Heartbeat lato client (Socket)

Al termine della fase di connessione, il `SocketClientNetworkAdapter` avvia il proprio thread di ascolto, che prima di entrare nel ciclo di elaborazione dei messaggi richiama `startClientTimeoutChecker()`. Questo metodo attiva uno scheduler che verifica se un `PING` è stato ricevuto entro **20 secondi**:

- Alla ricezione di un messaggio `PING` nel ciclo `processServerMessage()`, il client risponde *immediatamente* con un `PONG` e azzerava il timer interno.
- Se invece nessun `PING` viene ricevuto entro 20 secondi, lo scheduler invoca `onError()` sul `ClientController`, il quale mostra all'utente un messaggio di errore di timeout sulla UI e chiude la connessione.

La differenza di timeout (15 s lato server, 20 s lato client) introduce una piccola asimmetria intenzionale: il server è più reattivo nel liberare le risorse di partita, mentre il client attende un margine aggiuntivo prima di dichiarare la connessione persa, riducendo i falsi positivi dovuti a eventuali latenze di rete.

2.3 Heartbeat lato RMI

La logica di heartbeat per i client RMI segue gli stessi principi descritti per il canale Socket, con la differenza implementativa che sia il ping che la rilevazione del timeout sono realizzati tramite **chiamate a metodo remoto**: il server invoca periodicamente un metodo `ping()` sull'oggetto di callback del client (`RMIClientConnection`), e il client

risponde con un metodo `pong()` sull'oggetto remoto `RMIGameServer`. Un'eccezione RMI durante l'invocazione viene interpretata come segnale di disconnessione e innesca lo stesso meccanismo di `abortGame()` descritto sopra.

3 Sequence Diagram del Protocollo Client–Server

In questa sezione vengono descritti gli scenari principali del protocollo di comunicazione tra client e server.

(Per questioni di spazio le immagini dei sequence diagram non sono incluse nel seguente documento, ma si possono trovare nell'omonima cartella github)

Gli scenari considerati sono i seguenti:

1. **Connessione** al server e gestione delle **lobby**
2. **Avvio della partita** e invio del primo `GameStateUpdateMessage` (aggiornamento di stato del gioco)
3. Esecuzione di un'**azione di gioco** da parte del client
4. **Fine della partita** e invio del `RankingUpdateMessage` (classifica finale)
5. Meccanismo di heartbeat (**ping-pong**) e gestione dei timeout

3.1 Scenario 1: Connessione al server e gestione delle lobby

In questo scenario il focus è sulla creazione o sul join di una lobby e sul modo in cui il client stabilisce la connessione con il server tramite il protocollo selezionato (Socket o RMI), associandosi alla lobby corretta.

Lato client, l'utente avvia l'applicazione specificando da riga di comando interfaccia (TUI/GUI) e protocollo (Socket/RMI). Dopo l'avvio, l'utente sceglie se creare una nuova lobby oppure unirsi a una lobby esistente inserendo il codice univoco della stanza. A questo punto il `ClientController` richiede al `NetworkAdapterFactory` la creazione di un `ClientNetworkAdapter` specifico (Socket o RMI), che stabilisce effettivamente la connessione con il server.

Sul server, `HybridServerNetworkAdapter` riceve la nuova connessione (in modalità Socket tramite `SocketServerNetworkAdapter` e `ClientSocketHandler`, oppure in modalità RMI tramite `RMIServerNetworkAdapter` e `RMIGameServer`). Il server registra il nuovo giocatore all'interno della struttura di lobby gestita dal `MatchManager`, creando una nuova lobby con codice univoco oppure aggiungendo il giocatore a una lobby esistente, se il codice fornito è valido.

Infine, il server invia ai client interessati un `LobbyUpdateMessage` (lato Socket) o richiama le callback registrate (lato RMI) per notificare lo stato aggiornato della lobby (lista dei giocatori attualmente connessi), permettendo all'utente di vedere in tempo reale chi è presente nella stanza.

3.2 Scenario 2: Avvio della partita e primo GameStateUpdate

Questo scenario illustra cosa accade quando una lobby raggiunge il numero di giocatori necessario e la partita viene avviata dal server.

Il `MatchManager` monitora lo stato delle lobby e, al verificarsi delle condizioni di avvio (numero di giocatori raggiunto), invoca i metodi del `ServerController` per creare il modello di gioco iniziale (board, risorse, carte obiettivo, ecc.). Una volta creato lo stato iniziale, il `ServerController` costruisce uno snapshot consistente dello stato di gioco e lo incapsula in un `GameStateUpdateMessage`.

Lato rete, `HybridServerNetworkAdapter` riceve la richiesta di invio dell'aggiornamento e, per ciascun giocatore associato alla lobby, instrada il messaggio tramite il canale appropriato: via Socket usando `SocketServerNetworkAdapter` e `ClientSocketHandler`, oppure via RMI invocando le callback del `RMIClientConnection`. Ogni client riceve quindi il `GameStateUpdateMessage` iniziale, lo passa al proprio `ClientController` che aggiorna il modello locale e chiede alla UI (TUI/GUI) di renderizzare lo stato iniziale della partita.

3.3 Scenario 3: Azione di gioco del client

In questo scenario viene rappresentata l'esecuzione di una singola azione di gioco da parte del giocatore di turno, ad esempio la scelta di posizionare un totem su una tessera offerta (`placeTotemOnOfferTile`). Il flusso è rappresentativo anche di altre azioni di gioco, che seguono uno schema simile.

Lato client, l'utente interagisce con la UI (TUI/GUI) selezionando l'azione consentita dal turno corrente. La UI chiama il metodo corrispondente sul `ClientController`, che costruisce il messaggio di richiesta (ad esempio un oggetto comando o un messaggio contenente i parametri dell'azione) e lo invia al server tramite il `ClientNetworkAdapter` (Socket o RMI).

Sul server, `HybridServerNetworkAdapter` riceve il messaggio attraverso il canale specifico (Socket/RMI) e lo inoltra al `ServerController`. Quest'ultimo valida l'azione (controllando che il giocatore sia di turno, che le condizioni di gioco siano soddisfatte, ecc.) e, se l'azione è valida, aggiorna il modello di gioco (modificando board, risorse, tracciati, ecc.). Dopo l'aggiornamento, il `ServerController` genera un nuovo `GameStateUpdateMessage` e chiede a `HybridServerNetworkAdapter` di inviarlo a tutti i giocatori della partita.

Se invece l'azione non è valida (es. mossa illegale, risorse insufficienti), il server non aggiorna lo stato, ma invia al solo giocatore di turno un messaggio di errore tramite lo stesso canale di comunicazione. Il `ClientController` intercetta l'errore e chiede alla UI di mostrare un messaggio adeguato, senza alterare il modello locale di gioco.

3.4 Scenario 4: Fine partita e invio della classifica

Questo scenario descrive come il server determina la conclusione della partita, calcola i punteggi finali e invia la classifica ai client connessi, chiudendo poi correttamente le risorse di rete associate alla lobby.

Durante il gioco il `MatchManager` e il `ServerController` monitorano le condizioni di fine partita (numero di round). Quando viene rilevata la condizione finale (Fase di gioco =

"GAME_OVER") il **ServerController** calcola i punteggi dei giocatori applicando le regole di scoring del gioco e produce una rappresentazione ordinata della classifica finale.

Queste informazioni vengono incapsulate in un **RankingUpdateMessage** e passate a **HybridServerNetworkAdapter**, che le inoltra a tutti i giocatori della lobby, ancora una volta utilizzando il canale corretto per ciascun giocatore (Socket o RMI). Lato client, il **ClientController** riceve il messaggio, aggiorna il modello locale e chiede alla UI di mostrare la classifica finale e l'esito della partita (vincitore, punteggi, ecc.).

Dopo aver notificato la classifica, il server procede a chiudere la lobby (rimuovendola dal **MatchManager**) e a chiudere le connessioni di rete associate ai client di quella partita, permettendo al sistema di liberare le risorse (inclusi i nickname dei giocatori, che tornano nuovamente disponibili per nuove lobby).

3.5 Scenario 5: Meccanismo di heartbeat (ping-pong)

L'ultimo scenario rappresenta il meccanismo di heartbeat utilizzato per rilevare disconnessioni e problemi di rete, sia lato server che lato client. Qui si considera il canale Socket, ma la logica è analoga per RMI (con invocazioni remote al posto dei messaggi su socket).

Per ogni connessione Socket, il **ClientSocketHandler** lato server avvia **startHeartbeat()**, che ogni 5 secondi invia al client un messaggio PING e avvia un timer interno per attendere il PONG. Se il PONG arriva entro 15 secondi, il timer viene resettato e il ciclo riprende con un nuovo PING; se invece il timeout scade, il server chiude la connessione con quel client e, se la partita è in corso, invoca **abortGame()** sul **MatchManager**, generando le notifiche di partita terminata forzatamente per gli altri giocatori.

Sul lato client, il **SocketClientNetworkAdapter** avvia il thread di ascolto e lo scheduler di **startClientTimeoutChecker()**, che controlla periodicamente se un PING è stato ricevuto entro 20 secondi. Quando il client riceve un PING nel ciclo di **processServerMessage()**, risponde immediatamente con un PONG e aggiorna il timer interno. Se trascorrono 20 secondi senza aver ricevuto alcun PING, lo scheduler invoca **onError()** sul **ClientController**, che mostra un messaggio di errore di timeout all'utente e chiude la connessione.